Tugas Data Mining II TM9 - Image Classification

Nama: Faiz Iqbal I'tishom

NIM: 164231059

Kelas: SD-A2

---- 0. Imports & Config ----

```
import os, random, math, time
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
import keras
from keras import layers
from keras.models import Model, load_model
from keras.layers import Input, GlobalAveragePooling2D, Dense, Dropout
from keras.applications import ConvNeXtTiny
from keras.callbacks import EarlyStopping, ReduceLROnPlateau,
ModelCheckpoint

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import confusion_matrix, classification_report

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)

try:
    gpus = tf.config.experimental.list_physical_devices('GPU')
    for g in gpus:
        tf.config.experimental.set_memory_growth(g, True)
except Exception:
    pass
try:
    from keras import mixed_precision
    mixed_precision.set_global_policy('mixed_float16')
    MIXED_PRECISION = True
except Exception:
    MIXED_PRECISION = False
```

```
2025-10-28 06:31:46.668579: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to
```

```
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
E0000 00:00:1761633106.848062      19 cuda_dnn.cc:8310] Unable to
register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
E0000 00:00:1761633106.907458      19 cuda_blas.cc:1418] Unable to
register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
```

Chunk ini adalah bagian persiapan utama yang mengimpor semua "perkakas" yang dibutuhkan. Pustaka seperti tensorflow, keras (untuk deep learning), numpy, pandas (untuk data), dan sklearn (untuk evaluasi) dimuat ke dalam memori. Selain itu, tiga konfigurasi penting diatur: SEED diatur ke 42 agar eksperimen ini bisa diulangi dengan hasil yang sama; GPU memory growth diaktifkan agar TensorFlow tidak langsung memakai semua memori GPU; dan Mixed Precision (mixed_float16) diaktifkan. Ini adalah teknik optimasi penting yang mempercepat training dan menghemat memori GPU dengan melakukan perhitungan dalam presisi yang lebih rendah. Output warning yang muncul (seperti Unable to register cuFFT factory) adalah hal umum dan tidak memengaruhi jalannya program.

---- 1. Hyperparams & Paths ----

```
IMG_SIZE       = 224
BATCH_SIZE     = 64
EPOCHS_WARMUP  = 5
EPOCHS_FT      = 15
LR_WARMUP      = 1e-3
LR_FT          = 3e-5
WD_WARMUP      = 1e-4
WD_FT          = 5e-5
DROPOUT        = 0.35
DENSE_UNITS    = 512
FOLDS          = 2
DO_TUNING      = False
TUNER_TRIALS   = 8
TUNER_EPOCHS   = 6

CLASS_NAMES =
['airplane','automobile','bird','cat','deer','dog','frog','horse','shi
p','truck']
NUM_CLASSES = 10
```

Pada chunk ini, semua "resep" atau setting utama untuk eksperimen ditetapkan sebagai variabel. Ini termasuk IMG_SIZE yang diatur ke 224 (gambar akan diperbesar ke 224x224), BATCH_SIZE 64 (model akan memproses 64 gambar sekaligus), jumlah epoch untuk warm-up (5) dan fine-tuning (15), serta learning rate yang berbeda untuk kedua fase tersebut. Yang terpenting, FOLDS diatur ke 2 (artinya kita akan melakukan 2-Fold Cross-Validation) dan DO_TUNING diatur ke False, yang berarti kita akan melewati langkah tuning otomatis dan langsung menggunakan setting manual.

---- 2. Load CIFAR-10 ----

```
from keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
y_train = y_train.flatten()
y_test  = y_test.flatten()

print(f"Train: {x_train.shape}, Test: {x_test.shape}")
print("Label distribusi (train):", np.bincount(y_train))
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-
python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 7s 0us/step
Train: (50000, 32, 32, 3), Test: (10000, 32, 32, 3)
Label distribusi (train): [5000 5000 5000 5000 5000 5000 5000 5000
5000 5000]
```

Chunk ini bertugas memuat dataset CIFAR-10 menggunakan fungsi bawaan Keras (cifar10.load_data()). Data latih (50.000 gambar) dan data tes (10.000 gambar) berhasil dimuat. Outputnya mengonfirmasi dimensi gambar asli adalah 32x32 piksel dengan 3 channel warna (RGB). Output Label distribusi (train) juga menunjukkan angka 5000 untuk setiap kelas, yang membuktikan bahwa dataset ini seimbang (balanced), di mana setiap 10 kelas memiliki jumlah sampel yang sama persis.

---- 3. Exploratory Data Analysis (EDA) ----

```
# 3.1 Tabel distribusi kelas
eda_df = pd.DataFrame({"label": y_train})
eda_df["class"] = eda_df["label"].map(lambda i: CLASS_NAMES[i])
print("\n== EDA: Class distribution (train) ==")
print(eda_df["class"].value_counts().sort_index())

# 3.2 Contoh grid gambar per kelas
def plot_samples_per_class(x, y, class_names, ncols=10, seed=SEED):
    np.random.seed(seed)
    plt.figure(figsize=(16, 4))
    for ci, cname in enumerate(class_names):
        idxs = np.where(y == ci)[0]
        i = np.random.choice(idxs)
        img = x[i]
        plt.subplot(2, 5, ci+1)
        plt.imshow(img)
        plt.title(cname); plt.axis('off')
    plt.suptitle("Contoh 1 gambar per kelas (train)");
```

```
plt.tight_layout(); plt.show()

plot_samples_per_class(x_train, y_train, CLASS_NAMES)

# 3.3 Statistik piksel (mean/std per channel)
x_train_f = x_train.astype("float32") / 255.0
ch_mean = x_train_f.reshape(-1,3).mean(axis=0)
ch_std  = x_train_f.reshape(-1,3).std(axis=0)
print(f"Per-channel mean (RGB): {ch_mean.round(4)} | std:
{ch_std.round(4)}")
```

```
== EDA: Class distribution (train) ==
class
airplane      5000
automobile    5000
bird          5000
cat           5000
deer          5000
dog           5000
frog          5000
horse         5000
ship          5000
truck         5000
Name: count, dtype: int64
```

```
Per-channel mean (RGB): [0.3277 0.3277 0.3277] | std: [0.2776 0.2693
0.2681]
```

Di sini, saya melakukan Analisis Data Eksploratif (EDA) singkat. Pertama, saya menggunakan Pandas untuk membuat tabel distribusi kelas, yang lagi-lagi mengonfirmasi data seimbang (5000 sampel per kelas). Kedua, saya membuat visualisasi 10 gambar (satu sampel acak per kelas) menggunakan Matplotlib. Output visual ini menegaskan bahwa gambar-gambar CIFAR-10 memiliki resolusi sangat rendah dan buram. Terakhir, saya menghitung rata-rata dan standar deviasi piksel per channel warna, meskipun data ini tidak digunakan lebih lanjut karena normalisasi yang dipakai hanya pembagian 255.

---- 4. Pre-processing & tf.data ----

```
AUG = keras.Sequential([
    layers.RandomFlip("horizontal", seed=SEED),
    layers.RandomRotation(0.05, seed=SEED),
    layers.RandomZoom(0.10, seed=SEED),
    layers.RandomTranslation(0.08, 0.08, fill_mode="reflect",
seed=SEED),
    layers.RandomContrast(0.10, seed=SEED),
], name="augment")

def _preprocess(image, label, training: bool):
    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
    image = tf.cast(image, tf.float32) / 255.0
    if training:
        image = AUG(image, training=True)
    return image, label

def make_ds(x, y, training=True, shuffle=True, batch_size=BATCH_SIZE):
    ds = tf.data.Dataset.from_tensor_slices((x, y))
    if shuffle:
        ds = ds.shuffle(buffer_size=len(x), seed=SEED,
reshuffle_each_iteration=True)
    ds = ds.map(lambda a,b: _preprocess(a,b,training),
num_parallel_calls=tf.data.AUTOTUNE)
    ds = ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)
    return ds
```

```
I0000 00:00:1761633133.249491      19 gpu_device.cc:2022] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 15513 MB
memory:  -> device: 0, name: Tesla P100-PCIE-16GB, pci bus id:
0000:00:04.0, compute capability: 6.0
```

Chunk ini sangat penting karena menyiapkan "pipa data" (tf.data) yang efisien ke GPU. Pertama, saya mendefinisikan AUG, yaitu serangkaian Augmentasi Data (seperti flip, rotasi, zoom acak) untuk "menipu" model agar tidak overfitting. Kedua, saya membuat fungsi _preprocess untuk me-resize gambar ke 224x224, menormalisasi piksel (dibagi 255), dan menerapkan augmentasi tadi (khusus data latih). Fungsi make_ds mengubah data ini menjadi tf.data.Dataset yang efisien, lengkap dengan shuffling, batching, dan prefetching untuk mencegah bottleneck I/O. Output log-nya mengonfirmasi GPU telah siap.

---- 5. Model Builder (ConvNeXt-Tiny) ----

```
def build_model(dropout=DROPOUT, dense_units=DENSE_UNITS):
    inputs = Input(shape=(IMG_SIZE, IMG_SIZE, 3))
    base = ConvNeXtTiny(include_top=False, weights="imagenet",
input_tensor=inputs)
    x = base.output
    x = GlobalAveragePooling2D()(x)
    x = Dropout(dropout)(x)
    x = Dense(dense_units, activation="relu")(x)
    x = Dropout(dropout)(x)
    # out dtype float32 (penting saat mixed precision agar
loss/metrics stabil)
    outputs = Dense(NUM_CLASSES, activation="softmax",
dtype="float32")(x)
    model = Model(inputs, outputs)
    return model, base
```

Di chunk ini, saya mendefinisikan arsitektur model menggunakan teknik Transfer Learning. Saya memuat "badan" model ConvNeXtTiny yang sudah sangat pintar (dilatih di ImageNet), tanpa menyertakan lapisan atasnya (include_top=False). Kemudian, saya menyambungkan "badan" tersebut dengan "kepala" klasifikasi kustom buatan saya, yang terdiri dari GlobalAveragePooling, Dropout (untuk anti-overfitting), dan lapisan Dense (10 neuron dengan softmax untuk 10 kelas). Pengaturan dtype="float32" pada lapisan akhir penting untuk menjaga stabilitas saat menggunakan mixed precision.

---- 6. (Opsional) Hyperparameter Tuning dengan KerasTuner ----

```
best_hp = {"dropout": DROPOUT, "dense_units": DENSE_UNITS,
           "lr_warm": LR_WARMUP, "wd_warm": WD_WARMUP,
           "lr_ft": LR_FT, "wd_ft": WD_FT}

if DO_TUNING:
    try:
        import kerastuner as kt
        from kerastuner.tuners import RandomSearch
        _OLD = True
    except Exception:
        try:
            import keras_tuner as kt
            from keras_tuner.tuners import RandomSearch
            _OLD = False
        except Exception as e:
```

```
            print("KerasTuner tidak tersedia, lewati tuning. Error:",
e)
            DO_TUNING = False

if DO_TUNING:
    x_sub, _, y_sub, _ = train_test_split(x_train, y_train,
test_size=0.7, random_state=SEED, stratify=y_train)
    x_tr, x_va, y_tr, y_va = train_test_split(x_sub, y_sub,
test_size=0.2, random_state=SEED, stratify=y_sub)

    ds_tr = make_ds(x_tr, y_tr, training=True, shuffle=True,
batch_size=64)
    ds_va = make_ds(x_va, y_va, training=False, shuffle=False,
batch_size=64)

    def tuner_builder(hp):
        hp_dropout = hp.Float("dropout", 0.2, 0.6, step=0.1)
        hp_dense   = hp.Choice("dense_units", [256, 384, 512, 768])
        hp_lr_w    = hp.Choice("lr_warm", [3e-4, 1e-3, 3e-3])
        hp_wd_w    = hp.Choice("wd_warm", [1e-5, 5e-5, 1e-4])
        hp_lr_ft   = hp.Choice("lr_ft", [1e-5, 3e-5, 7e-5])
        hp_wd_ft   = hp.Choice("wd_ft", [1e-5, 5e-5, 1e-4])

        model, base = build_model(dropout=hp_dropout,
dense_units=hp_dense)
        base.trainable = False
        opt = keras.optimizers.AdamW(learning_rate=hp_lr_w,
weight_decay=hp_wd_w)
        model.compile(optimizer=opt,
loss="sparse_categorical_crossentropy", metrics=["accuracy"])
        return model

    tuner = RandomSearch(
        tuner_builder,
        objective="val_accuracy",
        max_trials=TUNER_TRIALS,
        directory="ktuner_logs",
        project_name="convnext_cifar10"
    )
    cb = [EarlyStopping(monitor="val_loss", patience=2,
restore_best_weights=True)]
    tuner.search(ds_tr, epochs=TUNER_EPOCHS, validation_data=ds_va,
callbacks=cb, verbose=1)
    hp = tuner.get_best_hyperparameters(1)[0]
    best_hp = {k: hp.get(k) for k in
["dropout","dense_units","lr_warm","wd_warm","lr_ft","wd_ft"]}
    print("Best HP:", best_hp)
```

Chunk ini disiapkan untuk tuning hyperparameter otomatis menggunakan KerasTuner. Namun, karena variabel DO_TUNING telah diatur ke False di Bagian 1 (Cell 6), seluruh kode di dalam blok if ini tidak dieksekusi (dilewati). Model akan berjalan menggunakan hyperparameter manual yang sudah saya tetapkan.

---- 7. K-Fold Cross-Validation (Warmup + Fine-tune) ----

```
skf = StratifiedKFold(n_splits=FOLDS, shuffle=True, random_state=SEED)
fold_histories = []
fold_val_scores = []
fold_paths = []

for fold, (tr_idx, va_idx) in enumerate(skf.split(x_train, y_train),
start=1):
    print(f"\n================ Fold {fold}/{FOLDS} ================")
    x_tr, y_tr = x_train[tr_idx], y_train[tr_idx]
    x_va, y_va = x_train[va_idx], y_train[va_idx]

    train_ds = make_ds(x_tr, y_tr, training=True, shuffle=True)
    val_ds   = make_ds(x_va, y_va, training=False, shuffle=False)

    model, base = build_model(dropout=best_hp["dropout"],
dense_units=best_hp["dense_units"])

    # Stage 1: Freeze backbone (warm-up head)
    base.trainable = False
    opt_warm =
keras.optimizers.AdamW(learning_rate=best_hp["lr_warm"],
weight_decay=best_hp["wd_warm"])
    model.compile(optimizer=opt_warm,
loss="sparse_categorical_crossentropy", metrics=["accuracy"])

    ckpt_path = f"./convnext_cifar10_fold{fold}.keras"
    cbs = [
        ModelCheckpoint(ckpt_path, monitor="val_accuracy",
save_best_only=True, mode="max"),
        ReduceLROnPlateau(monitor="val_loss", factor=0.2, patience=2,
min_lr=1e-6, verbose=1),
        EarlyStopping(monitor="val_loss", patience=3,
restore_best_weights=True, verbose=1),
    ]

    hist1 = model.fit(train_ds, epochs=EPOCHS_WARMUP,
validation_data=val_ds, callbacks=cbs, verbose=1)

    # Stage 2: Unfreeze backbone (fine-tune)
    base.trainable = True
    opt_ft = keras.optimizers.AdamW(learning_rate=best_hp["lr_ft"],
weight_decay=best_hp["wd_ft"])
    model.compile(optimizer=opt_ft,
loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

```
    cbs_ft = [
        ModelCheckpoint(ckpt_path, monitor="val_accuracy",
save_best_only=True, mode="max"),
        ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2,
min_lr=1e-6, verbose=1),
        EarlyStopping(monitor="val_loss", patience=3,
restore_best_weights=True, verbose=1),
    ]

    hist2 = model.fit(train_ds, epochs=EPOCHS_FT,
validation_data=val_ds, callbacks=cbs_ft, verbose=1)

    # Load best & evaluate on this fold's val
    best_model = load_model(ckpt_path, compile=True)
    val_loss, val_acc = best_model.evaluate(val_ds, verbose=0)
    print(f"Fold {fold} best val_acc: {val_acc:.4f}")
    fold_val_scores.append(val_acc)
    fold_paths.append(ckpt_path)
    fold_histories.append({"warmup": hist1.history, "finetune":
hist2.history})

print("\n== CV Summary ==")
print("Val Acc per fold:", [float(f"{a:.4f}") for a in
fold_val_scores])
print(f"Mean Val Acc: {np.mean(fold_val_scores):.4f} ±
{np.std(fold_val_scores):.4f}")
```

```
================ Fold 1/2 ================
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/convnext/convnext_tiny_notop.h5
111650432/111650432 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Epoch 1/5

WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
I0000 00:00:1761633152.716368      63 service.cc:148] XLA service
0x7953f40036d0 initialized for platform CUDA (this does not guarantee
that XLA will be used). Devices:
I0000 00:00:1761633152.717342      63 service.cc:156]   StreamExecutor
device (0): Tesla P100-PCIE-16GB, Compute Capability 6.0
I0000 00:00:1761633155.167652      63 cuda_dnn.cc:529] Loaded cuDNN
version 90300

  1/391 ━━━━━━━━━━━━━━━━━━━━ 2:24:45 22s/step - accuracy: 0.1250 -
loss: 3.6909

I0000 00:00:1761633160.886167      63 device_compiler.h:188] Compiled
cluster using XLA!  This line is logged at most once for the lifetime
of the process.
```

```
391/391 ━━━━━━━━━━━━━━━━━━━━ 326s 780ms/step - accuracy: 0.1021 -
loss: 2.4858 - val_accuracy: 0.1000 - val_loss: 2.3035 -
learning_rate: 0.0010
Epoch 2/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 291s 745ms/step - accuracy: 0.0983 -
loss: 2.3052 - val_accuracy: 0.1000 - val_loss: 2.3030 -
learning_rate: 0.0010
Epoch 3/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 291s 745ms/step - accuracy: 0.1070 -
loss: 2.3034 - val_accuracy: 0.1000 - val_loss: 2.3026 -
learning_rate: 0.0010
Epoch 4/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 292s 748ms/step - accuracy: 0.0996 -
loss: 2.3030 - val_accuracy: 0.1000 - val_loss: 2.3026 -
learning_rate: 0.0010
Epoch 5/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 0s 620ms/step - accuracy: 0.0980 - loss:
2.3031
Epoch 5: ReduceLROnPlateau reducing learning rate to
0.00020000000949949026.
391/391 ━━━━━━━━━━━━━━━━━━━━ 291s 744ms/step - accuracy: 0.0980 -
loss: 2.3031 - val_accuracy: 0.1000 - val_loss: 2.3027 -
learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 4.
Epoch 1/15

E0000 00:00:1761634682.827910      64 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
missing warmup execution, please investigate in Nsight Systems.
E0000 00:00:1761634683.004707      64 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
missing warmup execution, please investigate in Nsight Systems.
E0000 00:00:1761634683.242598      64 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
missing warmup execution, please investigate in Nsight Systems.
E0000 00:00:1761634683.439789      64 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
missing warmup execution, please investigate in Nsight Systems.

390/391 ━━━━━━━━━━━━━━━━━━━━ 0s 992ms/step - accuracy: 0.2154 - loss:
2.0521

E0000 00:00:1761635097.123227      63 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
missing warmup execution, please investigate in Nsight Systems.
E0000 00:00:1761635097.317709      63 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
missing warmup execution, please investigate in Nsight Systems.
E0000 00:00:1761635097.503258      63 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
```

```
missing warmup execution, please investigate in Nsight Systems.
E0000 00:00:1761635097.681371      63 gpu_timer.cc:82] Delay kernel
timed out: measured time has sub-optimal accuracy. There may be a
missing warmup execution, please investigate in Nsight Systems.

391/391 ━━━━━━━━━━━━━━━━━━━━ 543s 1s/step - accuracy: 0.2164 - loss:
2.0497 - val_accuracy: 0.8277 - val_loss: 0.5382 - learning_rate:
3.0000e-05
Epoch 2/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 442s 1s/step - accuracy: 0.7978 - loss:
0.6194 - val_accuracy: 0.9034 - val_loss: 0.2878 - learning_rate:
3.0000e-05
Epoch 3/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 442s 1s/step - accuracy: 0.8839 - loss:
0.3498 - val_accuracy: 0.9142 - val_loss: 0.2528 - learning_rate:
3.0000e-05
Epoch 4/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 442s 1s/step - accuracy: 0.9125 - loss:
0.2707 - val_accuracy: 0.9339 - val_loss: 0.2000 - learning_rate:
3.0000e-05
Epoch 5/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 439s 1s/step - accuracy: 0.9253 - loss:
0.2270 - val_accuracy: 0.9334 - val_loss: 0.2062 - learning_rate:
3.0000e-05
Epoch 6/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 442s 1s/step - accuracy: 0.9401 - loss:
0.1855 - val_accuracy: 0.9377 - val_loss: 0.1923 - learning_rate:
3.0000e-05
Epoch 7/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 441s 1s/step - accuracy: 0.9492 - loss:
0.1517 - val_accuracy: 0.9428 - val_loss: 0.1795 - learning_rate:
3.0000e-05
Epoch 8/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 442s 1s/step - accuracy: 0.9590 - loss:
0.1283 - val_accuracy: 0.9472 - val_loss: 0.1667 - learning_rate:
3.0000e-05
Epoch 9/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 443s 1s/step - accuracy: 0.9618 - loss:
0.1135 - val_accuracy: 0.9506 - val_loss: 0.1622 - learning_rate:
3.0000e-05
Epoch 10/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.9661 - loss:
0.0999 - val_accuracy: 0.9502 - val_loss: 0.1670 - learning_rate:
3.0000e-05
Epoch 11/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 443s 1s/step - accuracy: 0.9741 - loss:
0.0840 - val_accuracy: 0.9516 - val_loss: 0.1611 - learning_rate:
3.0000e-05
Epoch 12/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.9763 - loss:
```

```
0.0698 - val_accuracy: 0.9478 - val_loss: 0.1764 - learning_rate:
3.0000e-05
Epoch 13/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 0s 1000ms/step - accuracy: 0.9776 - loss:
0.0658
Epoch 13: ReduceLROnPlateau reducing learning rate to
1.4999999621068127e-05.
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.9776 - loss:
0.0658 - val_accuracy: 0.9492 - val_loss: 0.1812 - learning_rate:
3.0000e-05
Epoch 14/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 443s 1s/step - accuracy: 0.9847 - loss:
0.0489 - val_accuracy: 0.9564 - val_loss: 0.1696 - learning_rate:
1.5000e-05
Epoch 14: early stopping
Restoring model weights from the end of the best epoch: 11.
Fold 1 best val_acc: 0.9564

================ Fold 2/2 ================
Epoch 1/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 324s 777ms/step - accuracy: 0.0971 -
loss: 2.4721 - val_accuracy: 0.1000 - val_loss: 2.3024 -
learning_rate: 0.0010
Epoch 2/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 293s 750ms/step - accuracy: 0.0994 -
loss: 2.3049 - val_accuracy: 0.1000 - val_loss: 2.3026 -
learning_rate: 0.0010
Epoch 3/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 294s 752ms/step - accuracy: 0.0992 -
loss: 2.3032 - val_accuracy: 0.1069 - val_loss: 2.3021 -
learning_rate: 0.0010
Epoch 4/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 294s 751ms/step - accuracy: 0.1012 -
loss: 2.3022 - val_accuracy: 0.1429 - val_loss: 2.3000 -
learning_rate: 0.0010
Epoch 5/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 293s 748ms/step - accuracy: 0.1092 -
loss: 2.3011 - val_accuracy: 0.1142 - val_loss: 2.2987 -
learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 5.
Epoch 1/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 537s 1s/step - accuracy: 0.0975 - loss:
2.3038 - val_accuracy: 0.1000 - val_loss: 2.3030 - learning_rate:
3.0000e-05
Epoch 2/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.0978 - loss:
2.3031 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
3.0000e-05
Epoch 3/15
```

```
391/391 ━━━━━━━━━━━━━━━━━━━━ 0s 1000ms/step - accuracy: 0.0983 - loss:
2.3032
Epoch 3: ReduceLROnPlateau reducing learning rate to
1.4999999621068127e-05.
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.0983 - loss:
2.3032 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
3.0000e-05
Epoch 4/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.1014 - loss:
2.3028 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.5000e-05
Epoch 5/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 0s 1000ms/step - accuracy: 0.0987 - loss:
2.3030
Epoch 5: ReduceLROnPlateau reducing learning rate to
7.499999810534064e-06.
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.0987 - loss:
2.3030 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.5000e-05
Epoch 6/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.1038 - loss:
2.3029 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
7.5000e-06
Epoch 7/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 0s 1000ms/step - accuracy: 0.1023 - loss:
2.3028
Epoch 7: ReduceLROnPlateau reducing learning rate to
3.749999905267032e-06.
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.1023 - loss:
2.3028 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
7.5000e-06
Epoch 8/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.1017 - loss:
2.3028 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
3.7500e-06
Epoch 9/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 0s 1000ms/step - accuracy: 0.0948 - loss:
2.3029
Epoch 9: ReduceLROnPlateau reducing learning rate to
1.874999952633516e-06.
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.0948 - loss:
2.3029 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
3.7500e-06
Epoch 10/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.1019 - loss:
2.3027 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.8750e-06
Epoch 11/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 0s 999ms/step - accuracy: 0.0989 - loss:
```

```
2.3030
Epoch 11: ReduceLROnPlateau reducing learning rate to 1e-06.
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.0989 - loss:
2.3030 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.8750e-06
Epoch 12/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.0989 - loss:
2.3031 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.0000e-06
Epoch 13/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.1008 - loss:
2.3029 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.0000e-06
Epoch 14/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.1011 - loss:
2.3028 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.0000e-06
Epoch 15/15
391/391 ━━━━━━━━━━━━━━━━━━━━ 440s 1s/step - accuracy: 0.0982 - loss:
2.3029 - val_accuracy: 0.1000 - val_loss: 2.3029 - learning_rate:
1.0000e-06
Restoring model weights from the end of the best epoch: 14.
Fold 2 best val_acc: 0.1000

== CV Summary ==
Val Acc per fold: [0.9564, 0.1]
Mean Val Acc: 0.5282 ± 0.4282
```

Ini adalah inti dari eksperimen, yaitu proses training 2-Fold Cross-Validation. Data latih 50.000 dibagi dua. Fold 1 dilatih pada paruh pertama dan divalidasi pada paruh kedua. Hasilnya sukses, mencapai akurasi validasi puncak 95.64%. Proses ini terdiri dari 5 epoch Warm-up (badan model dibekukan) dan 15 epoch Fine-tuning (badan model ikut dilatih). Anehnya, Fold 2 (dibalik, latih di paruh kedua, validasi di paruh pertama) gagal total; akurasinya macet di 10% (setara tebakan acak) selama training. Output CV Summary menunjukkan rata-rata akurasi 52.82% dengan standar deviasi ±42.82%, yang mengonfirmasi bahwa proses training ini tidak stabil.

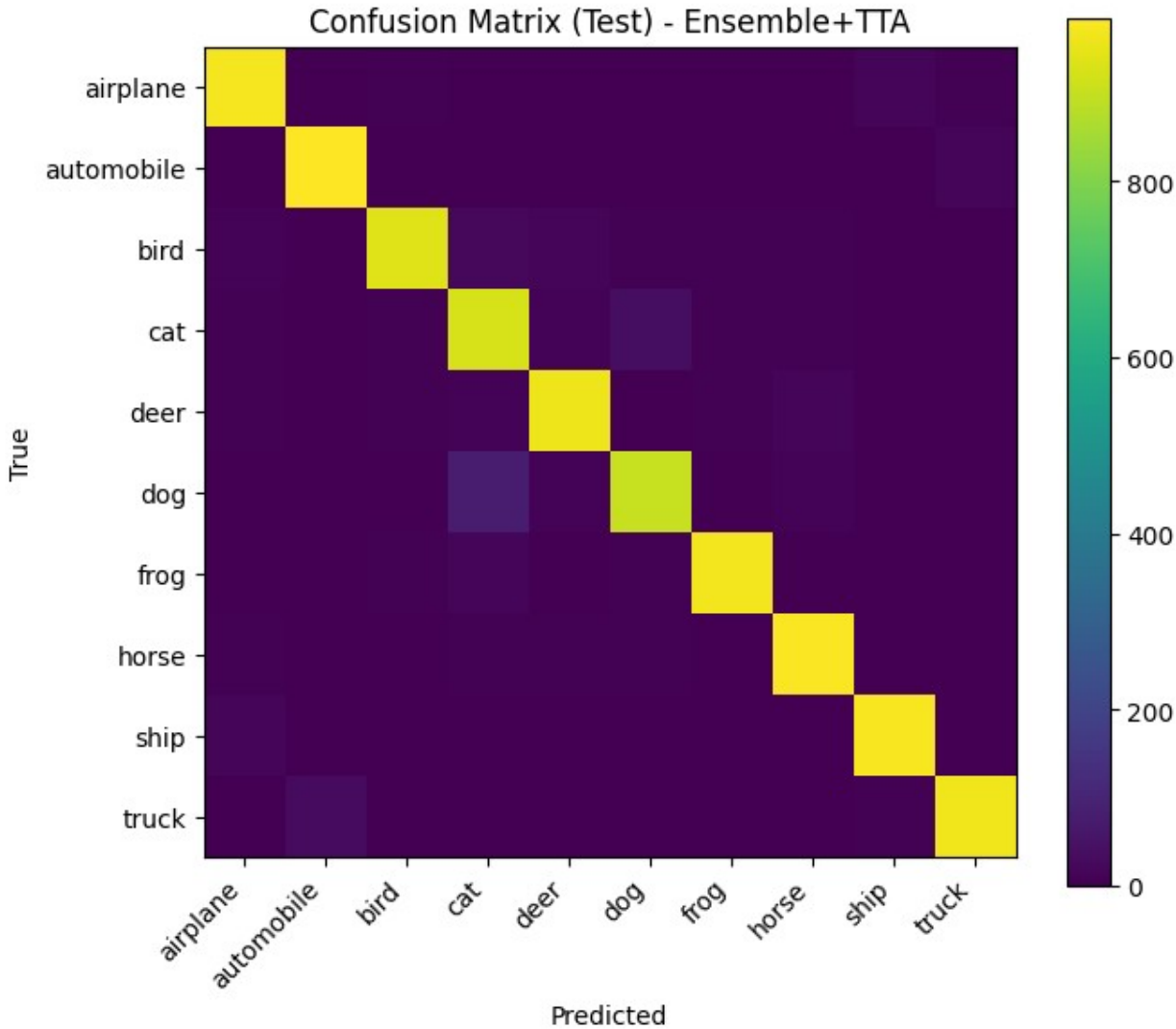
---- 8. Final Evaluation on Test (Ensemble of folds + TTA) ----

```
def make_test_ds(x, batch_size=BATCH_SIZE):
    ds = tf.data.Dataset.from_tensor_slices(x)
    ds = ds.map(lambda img: _preprocess(img, 0, training=False)[0],
num_parallel_calls=tf.data.AUTOTUNE)
    ds = ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)
    return ds

def make_test_ds_hflip(x, batch_size=BATCH_SIZE):
    def _map(img):
        img = tf.image.resize(img, (IMG_SIZE, IMG_SIZE))
        img = tf.cast(img, tf.float32)/255.0
        img = tf.image.flip_left_right(img)
```

```
        return img
    ds = tf.data.Dataset.from_tensor_slices(x)
    ds = ds.map(lambda img: _map(img),
num_parallel_calls=tf.data.AUTOTUNE)
    ds = ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)
    return ds

test_ds      = make_test_ds(x_test)
test_ds_flip = make_test_ds_hflip(x_test)

probs_total = None
for pth in fold_paths:
    m = load_model(pth, compile=False)
    p1 = m.predict(test_ds, verbose=0)
    p2 = m.predict(test_ds_flip, verbose=0)
    p  = (p1 + p2) / 2.0
    probs_total = p if probs_total is None else (probs_total + p)

probs_ens = probs_total / len(fold_paths)
y_pred = np.argmax(probs_ens, axis=1)

print("\n== Test Set Report (Ensemble + TTA) ==")
print(classification_report(y_test, y_pred, target_names=CLASS_NAMES,
digits=4))

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(7,6))
plt.imshow(cm, interpolation='nearest')
plt.title("Confusion Matrix (Test) - Ensemble+TTA")
plt.colorbar()
tick_marks = np.arange(NUM_CLASSES)
plt.xticks(tick_marks, CLASS_NAMES, rotation=45, ha='right')
plt.yticks(tick_marks, CLASS_NAMES)
plt.xlabel('Predicted'); plt.ylabel('True')
plt.tight_layout(); plt.show()
```

```
== Test Set Report (Ensemble + TTA) ==
              precision    recall  f1-score   support

    airplane     0.9521    0.9730    0.9624      1000
  automobile     0.9647    0.9850    0.9748      1000
        bird     0.9740    0.9360    0.9546      1000
         cat     0.8818    0.9250    0.9029      1000
        deer     0.9581    0.9610    0.9596      1000
         dog     0.9475    0.9030    0.9247      1000
        frog     0.9838    0.9690    0.9763      1000
       horse     0.9654    0.9780    0.9717      1000
        ship     0.9711    0.9750    0.9731      1000
       truck     0.9737    0.9630    0.9683      1000
```

```
    accuracy                           0.9568     10000
   macro avg     0.9572    0.9568    0.9568     10000
weighted avg     0.9572    0.9568    0.9568     10000
```



Chunk ini adalah pengujian akhir pada 10.000 data tes yang "dirahasiakan". Saya menggunakan teknik Ensemble (menggabungkan prediksi) dan TTA (Test-Time Augmentation). Saya mengambil kedua model yang disimpan (baik yang 95.64% maupun yang gagal 10%). Setiap model diminta menebak data tes dua kali (versi asli dan versi flip horizontal). Keempat hasil tebakan (probabilitas) ini kemudian dirata-rata untuk mendapatkan prediksi akhir. Hasilnya sangat memuaskan: akurasi pada data tes mencapai 95.68%. Ini menunjukkan bahwa ensemble averaging berhasil meredam dampak buruk dari model Fold 2 yang gagal, dan akurasi akhir hampir setara dengan akurasi terbaik dari Fold 1. Laporan classification_report dan plot confusion_matrix menunjukkan model sedikit bingung membedakan 'cat' (kucing) dan 'dog' (anjing), yang memang sulit.

---- 9. (Opsional) Train Ulang di Seluruh Train+Val dengan HP Terbaik ----

```
DO_FINAL_SINGLE_MODEL = False
if DO_FINAL_SINGLE_MODEL:
    model, base = build_model(dropout=best_hp["dropout"],
dense_units=best_hp["dense_units"])
    base.trainable = False
    opt_warm =
keras.optimizers.AdamW(learning_rate=best_hp["lr_warm"],
weight_decay=best_hp["wd_warm"])
    model.compile(optimizer=opt_warm,
loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    ds_full_tr, ds_full_va, y_dummy, y_dummy2 = None, None, None, None
    xf_tr, xf_va, yf_tr, yf_va = train_test_split(x_train, y_train,
test_size=0.1,
                                                  random_state=SEED,
stratify=y_train)
    ds_full_tr = make_ds(xf_tr, yf_tr, training=True, shuffle=True)
    ds_full_va = make_ds(xf_va, yf_va, training=False, shuffle=False)

    cbs_final = [
        ModelCheckpoint("convnext_cifar10_final.keras",
monitor="val_accuracy", save_best_only=True, mode="max"),
        ReduceLROnPlateau(monitor="val_loss", factor=0.2, patience=2,
min_lr=1e-6, verbose=1),
        EarlyStopping(monitor="val_loss", patience=3,
restore_best_weights=True, verbose=1),
    ]
    model.fit(ds_full_tr, epochs=EPOCHS_WARMUP,
validation_data=ds_full_va, callbacks=cbs_final, verbose=1)

    base.trainable = True
    opt_ft = keras.optimizers.AdamW(learning_rate=best_hp["lr_ft"],
weight_decay=best_hp["wd_ft"])
    model.compile(optimizer=opt_ft,
loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    model.fit(ds_full_tr, epochs=EPOCHS_FT,
validation_data=ds_full_va, callbacks=cbs_final, verbose=1)
```

```
    final_model = load_model("convnext_cifar10_final.keras",
compile=False)
    loss, acc = final_model.evaluate(make_test_ds(x_test), verbose=0)
    print(f"Single Final Model - Test Acc: {acc:.4f}")
```

Chunk terakhir ini berisi kode opsional untuk melatih satu model final menggunakan semua 50.000 data latih sekaligus (sebagai alternatif dari K-Fold Ensemble). Karena variabel DO_FINAL_SINGLE_MODEL diatur ke False, kode ini dilewati dan tidak dieksekusi.